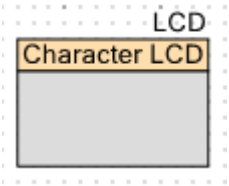
Lenguaje C - PSOC

✦ EMPEZAR CÓDIGO

```
#include "project.h"
int main(void) {
    CyGlobalIntEnable;

    for(;;) {}
}
```

✦ PANTALLA LCD



☆ *PINES DE CONEXIÓN:*

VDD → RL	*D7 → P2[3]*	*D6 → P2[2]*
DS → P2[1]	*D4 → P2[0]*	*E → P2[4]*
RW → P2[6]	*RS → P2[5]*	

☆ *Imprimir texto, (fila 0 a 3, posición 0 a 20)*

```
LCD_Position(0,0);
sprintf(textoLCD,"          Texto       ");
LCD_PrintString(textoLCD);
```

✦ LIBRERÍAS

- ***#include <stdio.h>*** *: Incluir las funciones de sprintf y scanf*
- ***#include <ctype.h>*** *: Permite leer la función isdigit*
- ***#include <stdbool.h>*** *: Permite usar booleanos*

✦ FUNCIONES

- ***CyGlobalIntEnable;*** *Habilita las interrupciones (detección temporal del programa) globales para que el programa pueda ser multitasking.*
- ***printf ("Mensaje %tipoVariable ", nombreVariable)*** *: Imprimir un mensaje concatenando los valores de una variable.*
- ***scanf("%tipoVariable", &nombreVariable);*** *Leer un número ingresado por el usuario*
- ***GetChar();*** *Leer comandos caracter por caracter.*
- ***Start();*** *Leer comandos caracter por caracter.*
- ***void*** *Nombre de la función* ***() { };*** *Me permite tener una función que no recibe ni entrega nada, se usa el 'void' para evitar errores.*
- ***//*** *Comentario de una (1) línea*
- ***\n*** *salto de línea*

✦ VARIABLES

- *char* ***%c*** *→ único carácter.*
- *int* ***%d*** *→ entero.*
- *float* ***%f*** *→ un número real en coma flotante, sin exponente.*
- *completar* ***%u*** *→ entero sin signo en base decimal.*

https://www.udec.cl/profile/13856/LAURITSHE?tab=collections

- *completar* **%o** → *entero en base octal.*
- *completar* **%x** → *entero en base hexadecimal.*
- *completar* **%e** → *un número real en coma flotante, con exponente.*
- *completar* **%s** → *cadena de carácteres.*
- *completar* **%p** → *puntero en dirección de memoria.*

✦ OPERADORES LÓGICOS

- *AND* **&&** → *ambas se cumplen.*
- *OR* **||** → *solo una se cumple.*
- *NOT* **!** → *se niega la expresión.*
- *A* **>=** *B* → *'A' mayor o igual que 'B'*
- *A* **<=** *B* → *'A' menor o igual que 'B'*
- *A* **==** *B* → *'A' igual que 'B'*

✦ NOTAS

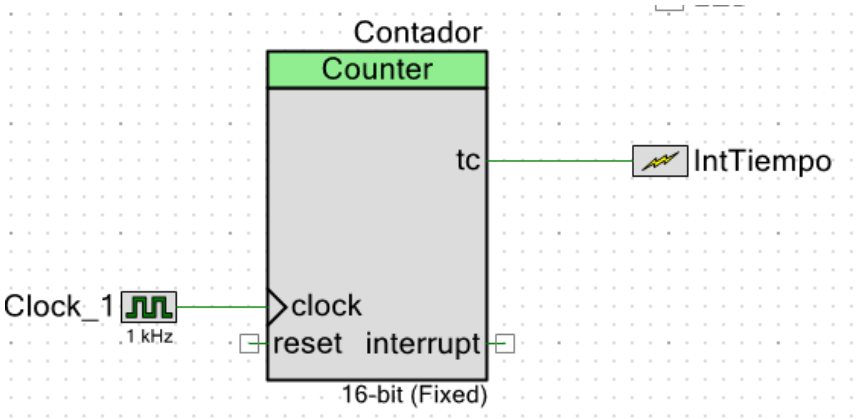
- ***Bandera*** → *Modificar el estado de una variable.*
- ***ISR*** → *Rutina de servicio de interrupciones.*
- ***Interrupciones*** → *Se arrancan con la función 'start execution'. Arrancan procesos ligeros (conocidos como* ***hilos****) que se pueden ejecutar en simultáneo con otros procesos.*
- ***LED Piloto*** → *Es una interrupción que tiene dos funciones, el recibir la información del serial e indicar el funcionamiento del PSOC, esto porque lee la función del LED y la niega instantáneamente haciendo que constantemente su valor oscile entre 0 y 1 (apagado y encendido).*
- ***Resetear bandera*** → *Una bandera se resetea cuando se pone en 0 después de que se hayan procesado sus respectivos eventos.*
- ***Contador*** → *Cuenta de manera decreciente (cuenta regresiva) según el periodo y la frecuencia configurada. Cada vez que el contador llega a 0, la terminal tc manda una señal de activación para que cada segundo se genere una interrupción (ISR).*
- *Pines del receptor y el transmisor ya están conectados físicamente a los pines P12[6] – receptor y P12[7]- transmisor.*

✦ CONTADOR.

☆ *Clock = $1kHz$, f=999 → 1 segundo*

☆ *Clock = $1kHz$, f=499 → 500 milisegundos*

☆ *Clock = $1kHz$, f=249 → 250 milisegundos*



https://www.udocz.com/profile/1956861/BURITSME?tab=collections

✦ SWITCH

```
switch (/*caracter de cambio*/) {
    case 'X':
    /* Instrucciones */;
    break;

    default:
    /* Si no se cumplen las opciones */;
    break;
}
```

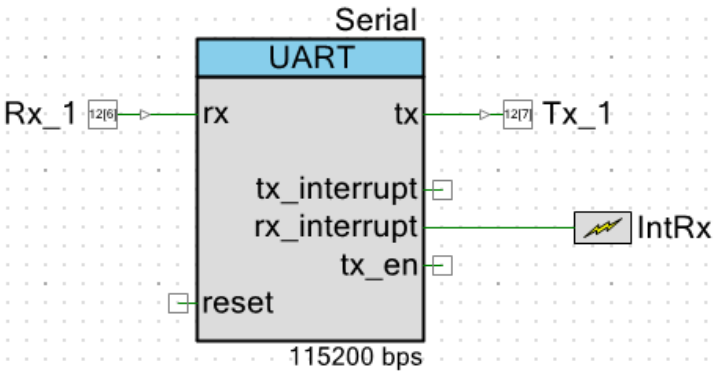
✦ IF

```
if (/*expresion logica*/){}
else {}
```

✦ WHILE

```
COMPLETAR
```

✦ RECIBIR CARÁCTER POR EL DOCKLIGHT




```
uint8 banderaDatoNuevo=0;
char caracterRx;

CY_ISR(RxData){
    banderaDatoNuevo=1;
    caracterRx=Serial_GetChar();
}

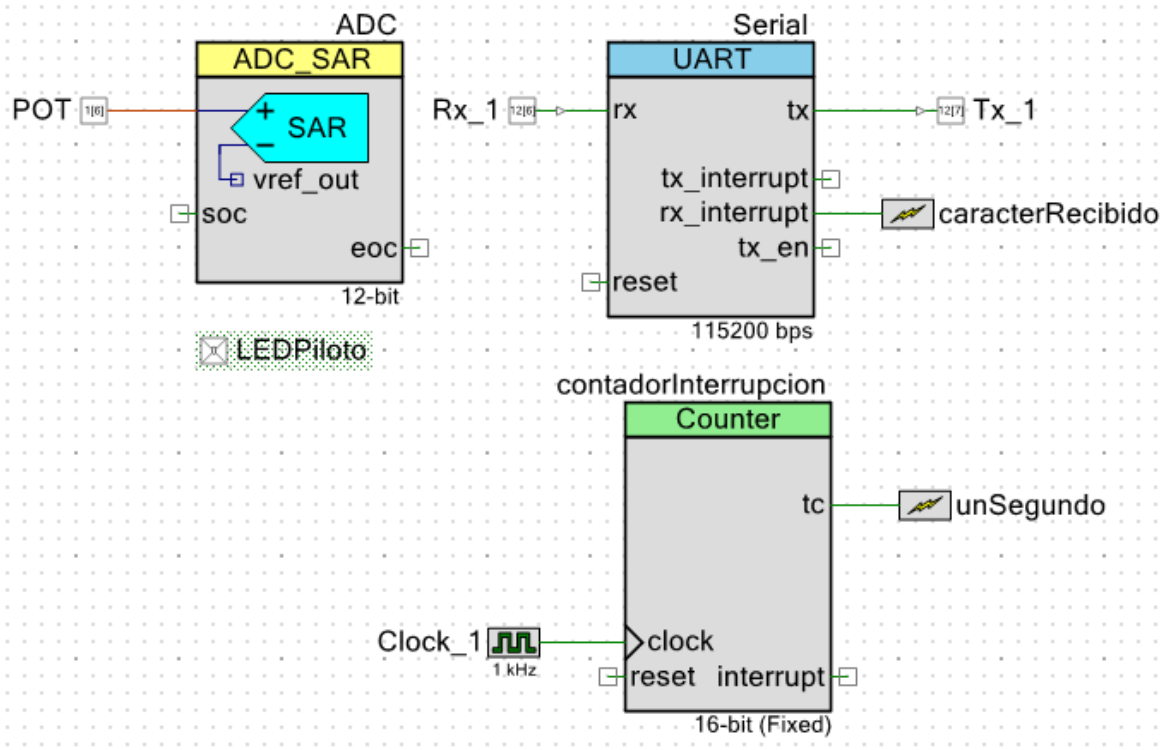

    int main(void){
    CyGlobalIntEnable;
    Contador_Start();
    IntTiempo_StartEx(UnSegundo);
    Serial_Start();
    IntRx_StartEx(RxData);

for(;;){
        if(banderaDatoNuevo==1){
            switch(caracterRx){}
banderaDatoNuevo=0;
```

http://www.ldocz.l/3586/LAURITSME?tab=collections

```
        }
}
```

✦ ACTUALIZAR POTENCIÓMETRO Y OBTENER SUS MV.




```
#include "project.h"
#include <stdio.h>

uint8 contadorTx=0;
uint8 banderaDatoRx=0;
uint8 banderaTiempo=0;
int16 datoADC=0, milivoltios=0;

char voltajePotenciometro;
char cadenaTexto[40];


CY_ISR(Recibir){
    banderaDatoRx=1;
    voltajePotenciometro=Serial_GetChar();
}

CY_ISR(interrupcionPiloto){
    banderaTiempo=1;
    LEDPiloto_Write(!LEDPiloto_Read());
}

void actualizarPotenciometro(){
    ADC_StartConvert();
    ADC_IsEndConversion(ADC_WAIT_FOR_RESULT);
    datoADC=ADC_GetResult16();
    milivoltios=ADC_CountsTo_mVolts(datoADC);
}

int main(void){
```

https://www.docz.com/profile/1... collections

```
    Serial_Start();
    ADC_Start();
    contadorInterrupcion_Start();

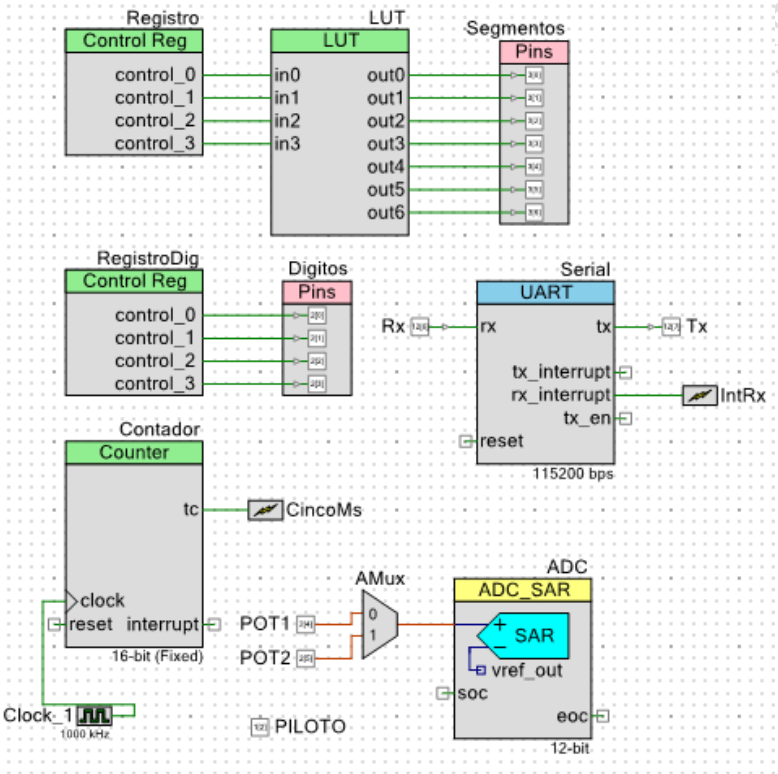
    caracterRecibido_StartEx(Recibir);
    unSegundo_StartEx(interrupcionPiloto);

    sprintf(cadenaTexto,"Esta miercoles funciona");
    Serial_PutString(cadenaTexto);

    CyGlobalIntEnable;

    for(;;){
        if(banderaTiempo==1){
            actualizarPotenciometro();
            sprintf(cadenaTexto,"Voltaje = %d mV\n",milivoltios);
            Serial_PutString(cadenaTexto);
            banderaTiempo=0;
            banderaDatoRx=0;
        }
    }
}
```

✦ DISPLAY 7 SEGMENTOS CON VOLTAJE HASTA 9999MV CON MUX




```
#include "project.h"
#include <stdio.h>

#define nContIntTiempo 20
uint8 Qdigito=1;
uint8 banderaSerial=0;
uint8 canalMux=0;
```

github.com/LAURITSME?tab=collections

```
uint8 banderaTiempo=0;
uint8 contadorInterrupcion=0;
uint16 datoADC, datoADCmV;

char cadena[60];
char CaracterRx;

typedef struct {
    uint16 numero;
    uint8 miles;
    uint8 centenas;
    uint8 decenas;
    uint8 unidades;
} numDisp;

numDisp num4Digitos;

void actualizarNumero() {
    num4Digitos.miles = (num4Digitos.numero / 1000) % 10;
    num4Digitos.centenas = (num4Digitos.numero / 100) % 10;
    num4Digitos.decenas = (num4Digitos.numero / 10) % 10;
    num4Digitos.unidades = num4Digitos.numero % 10;
    num4Digitos.numero=datoADCmV;
    if(num4Digitos.numero==10000){num4Digitos.numero=0;}
}

CY_ISR(datoSerial) {
    banderaSerial=1;
    CaracterRx=Serial_GetChar();
}

CY_ISR(IntTiempo){
    banderaTiempo=1;
}

int main(void){
    CyGlobalIntEnable;
    Contador_Start();
    CincoMs_StartEx(IntTiempo);
    ADC_Start();
    Serial_Start();
    IntRx_StartEx(datoSerial);
    AMux_Start();
    num4Digitos.numero=0;

    Serial_PutString(" Sistema para adquirir datos  \n");
    Serial_PutString(" Enviar A para mostrar el Canal 0, o B para Canal 1 \n");

    for(;;){
        if (banderaSerial==1){
            switch(CaracterRx){
                case 'A':
                canalMux=0;
                break;
                case 'B':
                canalMux=1;
                break;
                default:
                Serial_PutString("Opcion no valida, canal no existe \n");
                break;
```

https://www.udoz.com/profile/138586/LAURITSME?tab=collections

```
            }
            AMux_Select(canalMux);
            ADC_StartConvert();
            ADC_IsEndConversion(ADC_WAIT_FOR_RESULT);
            datoADC=ADC_GetResult16();
            datoADCmV=ADC_CountsTo_mVolts(datoADC);
            sprintf(cadena,"Canal:%d, Valor bruto:%d, V=%d mV \n",
canalMux,datoADC,datoADCmV);
            Serial_PutString(cadena);
            banderaSerial=0;
        }

        if(banderaTiempo==1){
            switch(Qdigito) {
                case 1:  // miles
                RegistroDig_Write(0b1000); // Activa transistor del D1
                Registro_Write(num4Digitos.miles);
                break;
                case 2: // centenas
                RegistroDig_Write(0b0100);
                Registro_Write(num4Digitos.centenas);
                break;
                case 3: // decenas
                RegistroDig_Write(0b0010);
                Registro_Write(num4Digitos.decenas);
                break;
                case 4: // unidades
                RegistroDig_Write(0b0001);
                Registro_Write(num4Digitos.unidades);
                break;
            }

            Qdigito++;

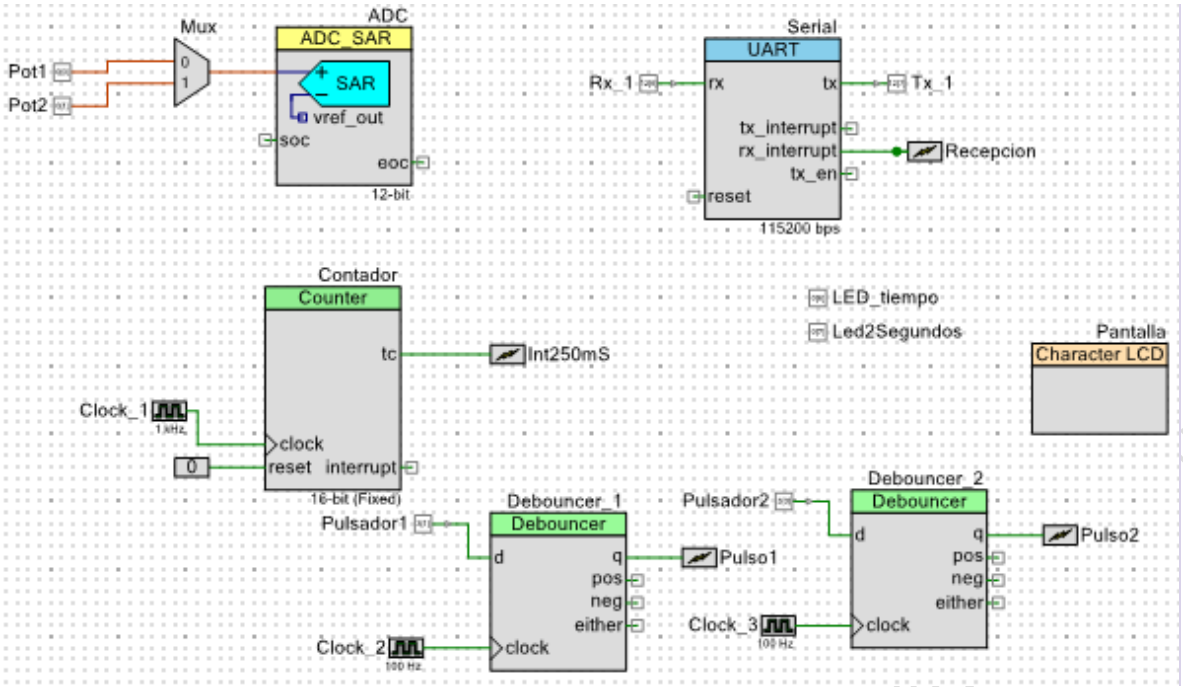
            if (Qdigito==5){Qdigito=1;}

            contadorInterrupcion++;
            banderaTiempo=0;
        }

            if (contadorInterrupcion==nContIntTiempo){
            /*Actualizar los dígitos que se muestra en la pantalla (En este ejemplo
también se incrementa cada vez que hay 20 interrupciones es decir cada 100
milisegundos)*/
            actualizarNumero();
            PILOTO_Write(!PILOTO_Read());
            contadorInterrupcion=0;
        }
    }
}
```

✦ LED PILOTO INTERMITENTE POR 250 MS, TRANSMITE EL VOLTAJE DE 2 POTENCIÓMETROS CON UN 'MUX', DOS BOTONES ANTI REBOTE Y PANTALLA LCD.

fritos://www.udemy.com/profile/13656/LAURITSME?tab=collections




```
#include "project.h"
#include <stdio.h>

int8 BanderaTiempo=0;
int8 BanderaRX=0;
int8 ContadorIntTiempo =0;
int8 BanderaP1=0;
int8 BanderaP2=0;
int16 resultadoADC=0;

char cadena1[50]="Los voy a corchar";
char caracterLeido='B';

CY_ISR(Tiempo){
    BanderaTiempo=1;
}

CY_ISR(Pul1){
    BanderaP1=1;
}

CY_ISR(Pul2){
    BanderaP2=1;
}

CY_ISR(nuevoDato){
    BanderaRX=1;
}

int main(void){
    CyGlobalIntEnable;
    ADC_Start();
    Serial_Start();
    Contador_Start();
    Pantalla_Start();
    Int250mS_StartEx(Tiempo);
    Recepcion_StartEx(nuevoDato);
```

https://www.udocz.com/profile/138586/LAUR... collections

```
Pulso1_StartEx(Pul1);
Pulso1_StartEx(Pul2);

Mux_Select(0);

for(;;){
    if (BanderaTiempo==1){
        LED_tiempo_Write(!LED_tiempo_Read());

        if (ContadorIntTiempo ==7){
            Led2Segundos_Write(!Led2Segundos_Read());

            if (Led2Segundos_Read()==0){
                Mux_Select(0);
                ADC_StartConvert();
                ADC_IsEndConversion(ADC_WAIT_FOR_RESULT);
                resultadoADC=ADC_GetResult16();
                resultadoADC=ADC_CountsTo_mVolts(resultadoADC);
                sprintf(cadena1,"Volt Pot1= %d mV \n",resultadoADC );
                Serial_PutString(cadena1);
                Pantalla_Position(0,1);
                Pantalla_PrintString(cadena1);
            }
            else{
                Mux_Select(1);
                ADC_StartConvert();
                ADC_IsEndConversion(ADC_WAIT_FOR_RESULT);
                resultadoADC=ADC_GetResult16();
                resultadoADC=ADC_CountsTo_mVolts(resultadoADC);
                sprintf(cadena1,"Volt Pot2= %d mV \n",resultadoADC );
                Serial_PutString(cadena1);
                Pantalla_Position(0,1);
                Pantalla_PrintString(cadena1);
            }

        }
        ContadorIntTiempo++;
        if (ContadorIntTiempo ==8){
            ContadorIntTiempo =0;
        }
        BanderaTiempo=0;
    }

    if (BanderaP1==1){
        Mux_Select(0);
        ADC_StartConvert();
        ADC_IsEndConversion(ADC_WAIT_FOR_RESULT);
        resultadoADC=ADC_GetResult16();
        resultadoADC=ADC_CountsTo_mVolts(resultadoADC);
        sprintf(cadena1,"Volt Pot1= %d mV \n",resultadoADC );
        Serial_PutString(cadena1);
        BanderaP1=0;
    }

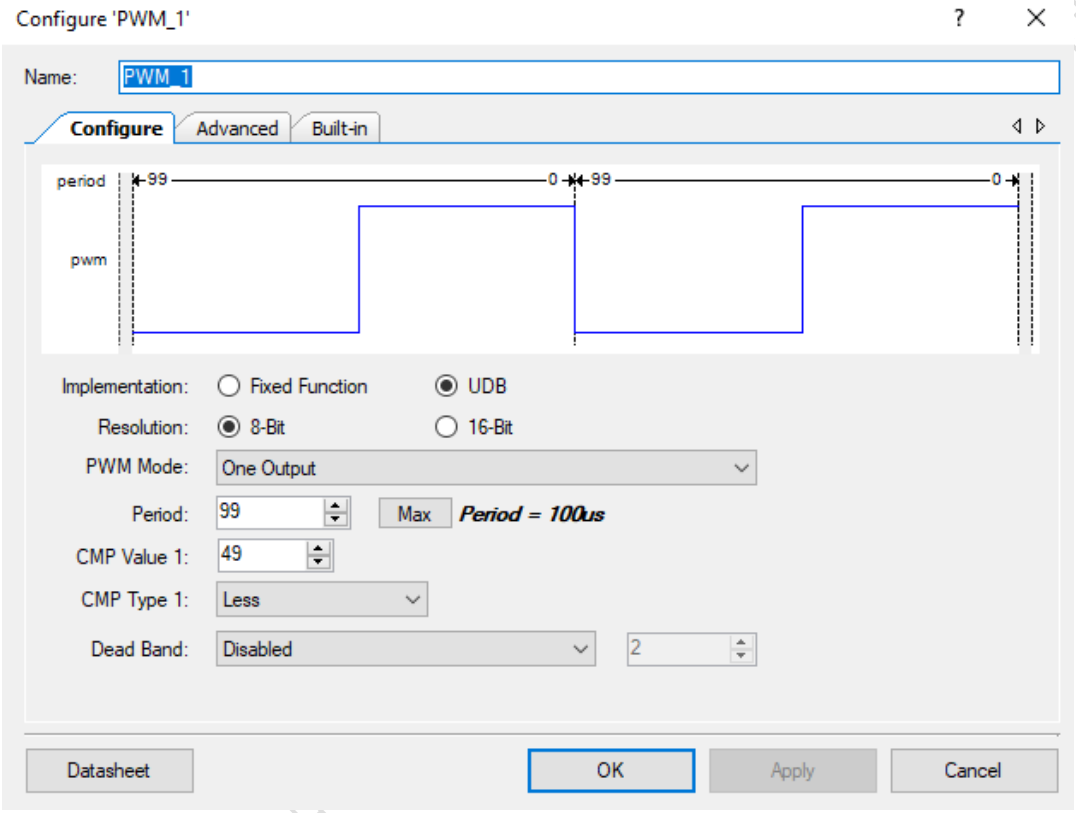
    if(BanderaP2==1){
        Mux_Select(1);
        ADC_StartConvert();
        ADC_IsEndConversion(ADC_WAIT_FOR_RESULT);
        resultadoADC=ADC_GetResult16();
        resultadoADC=ADC_CountsTo_mVolts(resultadoADC);
```

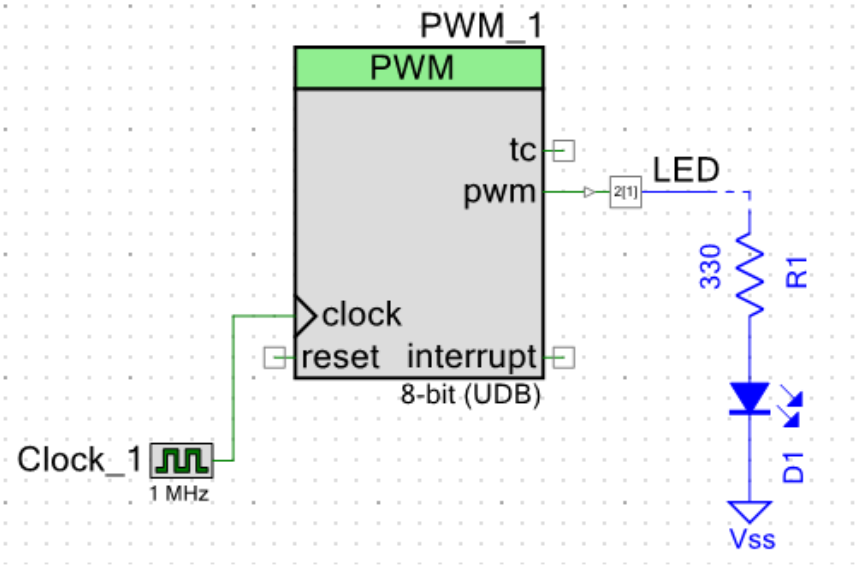
https://www.udocz.com/profile/LAURITSME?tab=collections

```
            sprintf(cadena1,"Volt Pot2= %d mV \n",resultadoADC );
            Serial_PutString(cadena1);
            BanderaP2=0;
        }

        if (BanderaRX==1){
            caracterLeido=Serial_GetChar();
            BanderaRX=0;
        }
    }
}
```

✦ USO DEL PWM: ESTE PROGRAMA VA VARIANDO LA INTENSIDAD DEL LED CONECTADO EN P2.1 DESDE 0% DE POTENCIA HASTA 99% DE LA POTENCIA, VARIANDO CADA SEGUNDO.





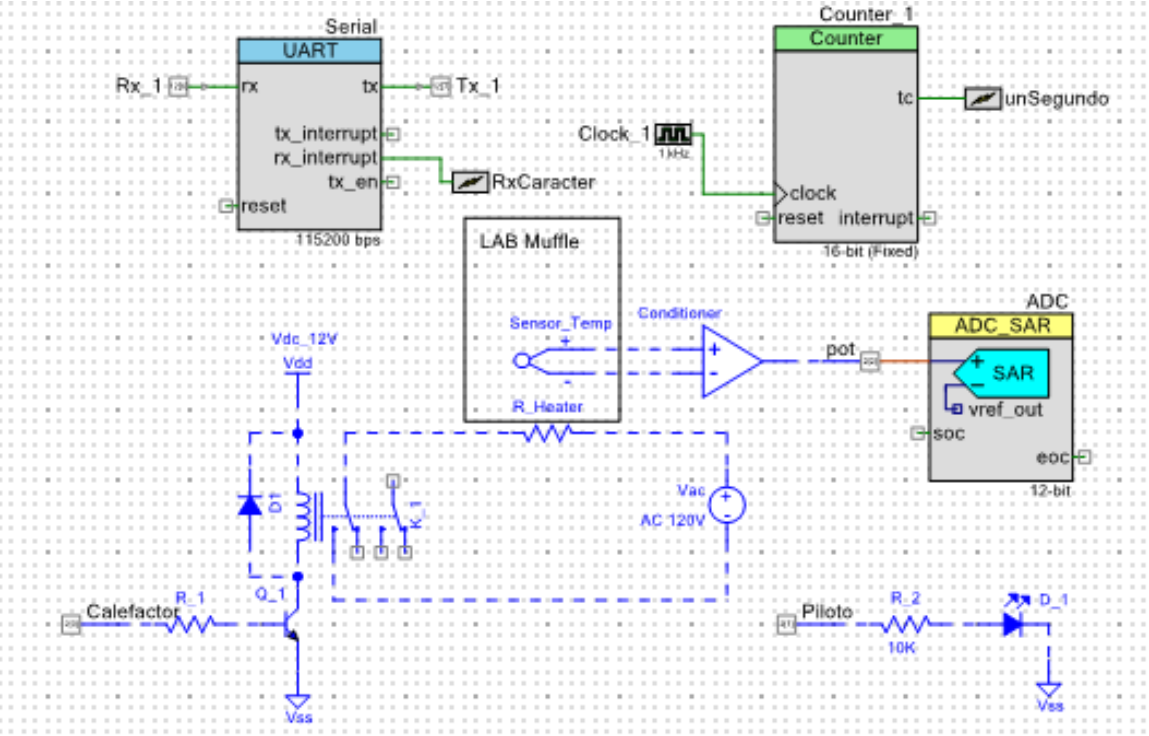

```c
#include "project.h"

int main(void){
    CyGlobalIntEnable;
    PWM_1_Start();

    for(;;){
        PWM_1_WriteCompare(0);   //Duty =0%, LED apagado
        CyDelay(1000);           //Retardo de 1000mSeg = 1 Seg
        PWM_1_WriteCompare(25); //Duty =25%, LED a 1/4 del máximo
        CyDelay(1000);
        PWM_1_WriteCompare(50); //Duty =50%, LED a 1/2 del máximo
        CyDelay(1000);
        PWM_1_WriteCompare(75); //Duty =75%, LED a 3/4 del máximo
        CyDelay(1000);
        PWM_1_WriteCompare(99); //Duty =100%, LED al máximo con esta
        CyDelay(1000);           //configuracion
    }
}
```

✦ MÁQUINA DE ESTADOS → MUFLA

https://www.udocz.com/profile/1005586/LAURITSME?tab=collections




```
#include "project.h"
#include <stdio.h>

#define TEMP_UPPER_LIMIT 800
#define TEMP_LOWER_LIMIT 700

typedef enum {
    INICIO,
    CONTROLANDO_TEMP,
    FINALIZANDO
} Estado;

Estado estadoActual = INICIO;
uint8 contadorTx=0;
uint8 banderaDatoRx=0;
uint8 banderaTiempo=0;
uint8 heaterOn = 0;  // Variable para rastrear el estado del calefactor
(0=apagado)
uint16 datoADC, temperaturaHorno;

char datoRX;
char cadenaTexto[40]="Arrancando el sistema!\n";

CY_ISR(Recibir){
    banderaDatoRx=1;
    datoRX=Serial_GetChar();
}

CY_ISR(baseDeTiempo){
    banderaTiempo=1;
    Piloto_Write(!Piloto_Read());       //piloto intermitente
```

https://www.udemy.com/profile/13856/Lin...=collections

```
}

void actualizarSensores(){
    ADC_StartConvert();
    ADC_IsEndConversion(ADC_WAIT_FOR_RESULT);
    datoADC=ADC_GetResult16();
    temperaturaHorno=(1000*datoADC)/4095;
}

void actualizarActuadores(){
    if (!heaterOn && temperaturaHorno < TEMP_LOWER_LIMIT) {
        Calefactor_Write(1);  // Si la temperatura es menor que el límite
inferior y el calefactor está apagado, enciende el calefactor
        heaterOn = 1;
    }
    else if (heaterOn && temperaturaHorno >= TEMP_UPPER_LIMIT) {
        Calefactor_Write(0);  // Si la temperatura es mayor o igual al límite
superior y el calefactor está encendido, apaga el calefactor
        heaterOn = 0;
    }
}

int main(void){
    Serial_Start();
    RxCaracter_StartEx(Recibir);
    Serial_PutString(cadenaTexto);
    unSegundo_StartEx(baseDeTiempo);
    Counter_1_Start();
    ADC_Start();

    CyGlobalIntEnable;
    for(;;){
       switch(estadoActual) {
            case INICIO:
                if(banderaTiempo==1){
                    sprintf(cadenaTexto,"Temperatura Horno: %d oC
\n",temperaturaHorno);
                    Serial_PutString(cadenaTexto);
                    actualizarSensores();
                    banderaTiempo=0;
                }
                if (datoRX=='A')
                    estadoActual = CONTROLANDO_TEMP;
                else
                    estadoActual = INICIO;
                break;
            case CONTROLANDO_TEMP:
                if(banderaTiempo==1){
                    sprintf(cadenaTexto,"Control Horno T= %d oC
\n",temperaturaHorno);
                    Serial_PutString(cadenaTexto);
                    actualizarSensores();
                    actualizarActuadores();
                    banderaTiempo=0;
```

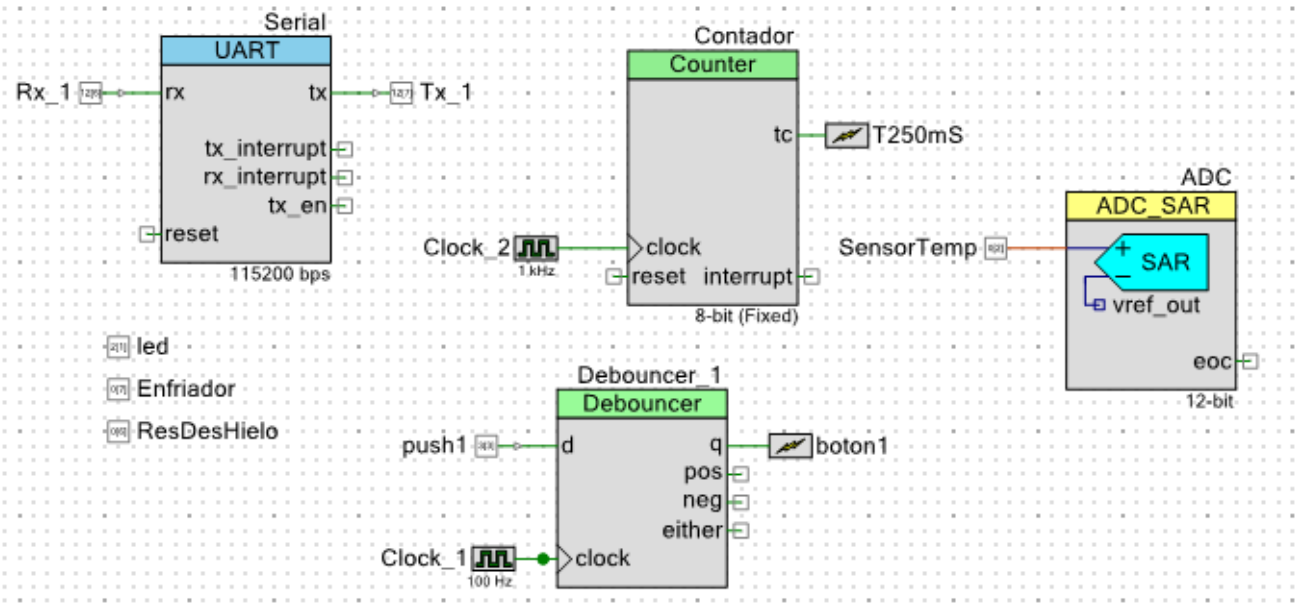
https://www.udocz.com/profile/138586/LAURITSMEY?tab=collections

```
                }
                if(datoRX!='P'){
                    estadoActual = CONTROLANDO_TEMP;
                }
                else
                    estadoActual = FINALIZANDO;
                break;
            case FINALIZANDO:
                Serial_PutString("Fin del proceso");
                Calefactor_Write(0);
                while(1);

                break;
            default:
                break;
        }

    }
}
```

✦ REFRIGERADOR DE MEDICAMENTOS: USO DE MÁQUINAS DE ESTADOS Y *DEBOUNCER*



MAIN.C

```
#include "project.h"
#include <stdio.h>
#include "lemox.h"
// Define el tiempo para iniciar el descongelamiento
#define ContadorDescongelar 480

// Enumeración para gestionar los estados del sistema de la nevera
typedef enum {
    esperando,
    controlTemp,
    descongelando,
    aperturaPuerta,
    alarma
} estadoSistema;
```

```
uint8 banderaBoton1=0;
uint8 banderaTiempo=0;
uint8 banderaMsgControl=1;
uint8 banderaEnfriador=0;
uint16 contadorDescong=0;
uint8 contDesHielo=0;
int8 temp;
char mensaje[50];
estadoSistema estadoNevera=esperando;

CY_ISR(intCuartoSegundo){
    banderaTiempo=1;
}

CY_ISR(control1){
    banderaBoton1=1;
}

int main(void){
    CyGlobalIntEnable;
    ADC_Start();
    Serial_Start();
    boton1_StartEx(control1);
    Serial_PutString("Sistema de Nevera de Medicamentos\n");
    Contador_Start();
    T250mS_StartEx(intCuartoSegundo);
    apagar_led();

    for(;;){
        switch(estadoNevera){
            case(esperando):
            if(banderaTiempo==1){
                Blink_Led();
                banderaTiempo=0;
            }
            // Cambio de estado por botón
            if(banderaBoton1==1){
                estadoNevera = controlTemp;
                banderaBoton1=0;
                prender_led();
            }
            break;
            // Controla la temperatura y el sistema de enfriamiento
            case(controlTemp):
            if (banderaTiempo==1){
                if(banderaMsgControl==1){
                    Serial_PutString("Pasando a Enfriar! \n");
                    banderaEnfriador =1;
                    banderaMsgControl=0;
                }
                temp=medirTemperatura();
                banderaEnfriador=contTemp(temp,banderaEnfriador);
                sprintf(mensaje,"T=%doC Comp=%d\n",temp,Enfriador_Read());
                Serial_PutString(mensaje);
                contadorDescong++;
                if(contadorDescong==ContadorDescongelar){
                    estadoNevera = descongelando;
                    contadorDescong=0;
```

https://www.coursehero.com/file/138586/EAURITSME?tab=collections

```
                }
                banderaTiempo=0;
            }
            break;
            // Gestiona el ciclo de descongelación
            case(descongelando):
            if (banderaTiempo==1)
            {
                Serial_PutString("Descongelar \n");
                ResDesHielo_Write(1);
                Blink_Led();
                contDesHielo++;
                if(contDesHielo==30)
                {
                    estadoNevera=controlTemp;
                    contDesHielo=0;
                    ResDesHielo_Write(0);
                }
                banderaTiempo=0;
            }
            break;
            case(aperturaPuerta):
            // Estado para gestionar alarmas (sin implementar)
            break;
            // Estado para gestionar alarmas (sin implementar)
            case(alarma):
            break;
        }
    }
}
```

LEMOX.C

```
#include "project.h"
#include "lemox.h"

void prender_led(){
    led_Write(1);
}

void apagar_led(){
    led_Write(0);
}

void Blink_Led(){
    led_Write(!led_Read());
}

int8 medirTemperatura(){
    ADC_StartConvert();
    ADC_IsEndConversion(ADC_WAIT_FOR_RESULT);
    uint16 resultado = ADC_GetResult16();
    uint16 resultadoMV = ADC_CountsTo_mVolts(resultado);
    int8 temperatura = resultadoMV/100-10;
    return temperatura;
}

uint8 contTemp(int8 temperatura , uint8 enfriadorEncendido){
  if (!enfriadorEncendido && temperatura > 8) {
```

https://www.udocz.com/profile/138586/LAURITSME?tab=collections

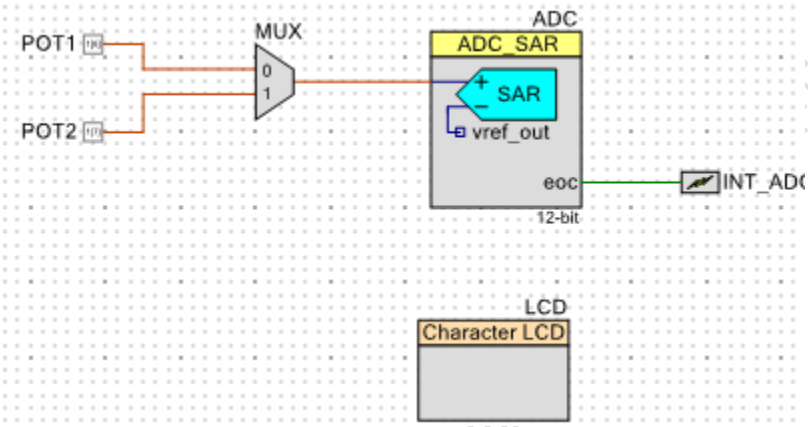
```
        enfriadorEncendido = 1;
        Enfriador_Write(1);
    }
    else if (enfriadorEncendido && temperatura < 2) {
        enfriadorEncendido = 0;
        Enfriador_Write(0);
    }
    return enfriadorEncendido;
}
```

LEMOX.H

```
#include "project.h"
void prender_led(void);
void apagar_led(void);
void Blink_Led(void);

int8 medirTemperatura(void);
uint8 contTemp(int8, uint8);
```

✦ ADQUISICIÓN DE DATOS DE DOS POTENCIÓMETROS Y MOSTRARLOS EN PANTALLA LCD




```
#include "project.h"
#include <stdio.h>

uint8 banderaADC=0;
uint8 canal=0;
uint816 datoADC;
char texto[40];

CY_ISR(conversionADC){
      banderaADC=1;
      datoADC=ADC_GetResult16();
}

int main(void){
      CyGlobalIntEnable;
      LCD_Start();
      ADC_Start();
      MUX_Start();
      INT_ADC_StartEx(conversionADC);
      ADC_StartConvert();
      LCD_Position(0,0);
```

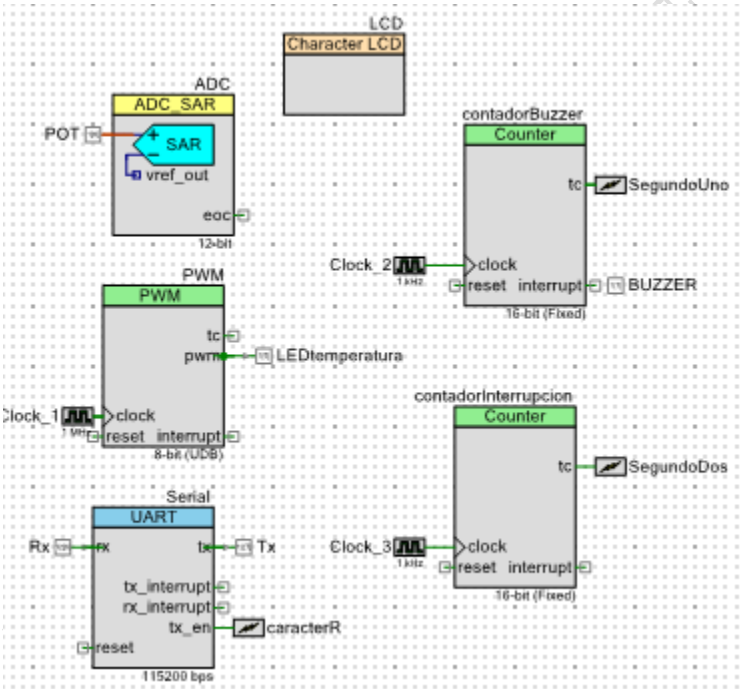
https://www.udocz.com/LAURITSME?tab=collections

```
        LCD_PrintString("Hola Mundo");

        for(;;) {
              if(banderaADC==1){
                    if(canal==0){
                    LCD_Position(1,0);
                    sprintf(texto,"Lectura Canal:%d=%d\n",canal,datoADC);
                    LCD_PrintString(texto);
                    canal = 1;
                    }
              else{
                    LCD_Position(2,0);
                    sprintf(texto,"Lectura Canal:%d=%d\n",canal,datoADC);
                    LCD_PrintString(texto);
                    canal = 0;
              }
              MUX_Select(canal);
              ADC_StartConvert();
              banderaADC = 0;
              }
        }
}
```

✦ SENSOR DE TEMPERATURA, PROPORCIONANDO UNA SEÑAL DE 100MV POR GRADO CENTÍGRADO

-Si la temperatura es menor a 30°C, en el LED veremos una alta intensidad
-Si la temperatura es mayor a 42°C, el LED manejara una baja intensidad lumínica
-Para el rango faltante, el buzzer sonará de forma intermitente por 20 segundos y tanto en Docklight como en el LCD muestra el valor de la temperatura, el valor del PWM y si el buzzer se activa o no




```
#include "project.h"
#include <stdio.h>

uint8 banderaLED=0;
uint8 banderaBuzzer=0;
```

```
uint8 banderaSerial=0;
uint8 off=0;
uint16 Temp=0;
uint16 datoADC=0;
uint16 mvoltios = 0;

char voltaje;
char texto[100];
char textobuzzer[100];
char textoLCD[100];

CY_ISR (intSegundoUno){
    banderaLED=1;
}

CY_ISR (intSegundoDos){
    banderaBuzzer=1;
}

CY_ISR (potenciometroSerial){
      banderaSerial=1;
      voltaje=Serial_GetChar();
}

void actualizarPotenciometro () {
      ADC_StartConvert();
      datoADC=ADC_GetResult16();
      mvoltios=ADC_CountsTo_mVolts(datoADC);
    Temp=mvoltios/100;
}

void limpiarLCD(){
    LCD_Position(0,0);
    sprintf(textoLCD,"                                                ");
    LCD_PrintString(textoLCD);
    LCD_Position(1,0);
    sprintf(textoLCD,"                                                ");
    LCD_PrintString(textoLCD);
    LCD_Position(2,0);
    sprintf(textoLCD,"                                                ");
    LCD_PrintString(textoLCD);
    LCD_Position(3,0);
    sprintf(textoLCD,"                                                ");
    LCD_PrintString(textoLCD);
}

int main(void){
    CyGlobalIntEnable;
    LCD_Start();
    contadorInterrupcion_Start();
    contadorBuzzer_Start();
    SegundoDos_StartEx(intSegundoDos);
    SegundoUno_StartEx(intSegundoUno);
    Serial_Start();
    ADC_Start();
    PWM_Start();
    caracterR_StartEx(potenciometroSerial);
```

https://www.scribd.com/profile/13856/LAURITSME?tab=collections

```
limpiarLCD();
LCD_Position(0,0);
LCD_PrintString("Iniciado el programa");

Serial_PutString("\nPrograma Iniciado\n");

for(;;){
    if(banderaLED==1) {
        sprintf(texto, "VOLTAJE = %d mV; TEMPERATURA = %d C \n", mvoltios, Temp);
        limpiarLCD();
        LCD_Position(1,0);
        sprintf(textoLCD,"VOL:%dmV; TEMP:%dC ",mvoltios, Temp);
        LCD_PrintString(textoLCD);
        Serial_PutString(texto);
        actualizarPotenciometro();

        if(Temp<=30){
                PWM_WriteCompare(99);
                BUZZER_Write(0);
            }

        banderaLED=0;

        if (Temp>=42){
                PWM_WriteCompare(15);
                BUZZER_Write(0);
            }

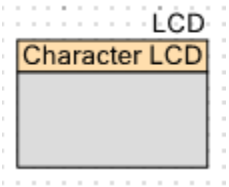
        banderaLED=0;
        if (Temp>30 && Temp<42){
                sprintf(texto,"\nBuzzer Activado\n");
                Serial_PutString(texto);
                limpiarLCD();
                LCD_Position(1,0);
                sprintf(textoLCD,"Buzzer activado");
                LCD_PrintString(textoLCD);
                if(banderaBuzzer==1){
                    BUZZER_Write(!BUZZER_Read());
                    sprintf(textobuzzer,"Conteo: %d seg",off);
                    Serial_PutString(textobuzzer);
                    off++;
                    banderaBuzzer=0;
                }
                    if(off==20){
                        BUZZER_Write(0);
                        sprintf(texto,"\nBuzzer Desactivado. Limite de tiempo\n");
                        Serial_PutString(texto);
                        limpiarLCD();
                        LCD_Position(1,0);
                        sprintf(textoLCD,"Buzzer desactivado");
                        LCD_PrintString(textoLCD);
                        off=0;
                    }
                }
            }

    banderaLED=0;
    }
}
```

https://www.udocz.com/read/13586/LAURITSME?tab=collections

```
}
```

✦ LIMPIAR PANTALLA LCD E IMPRIMIR TEXTO



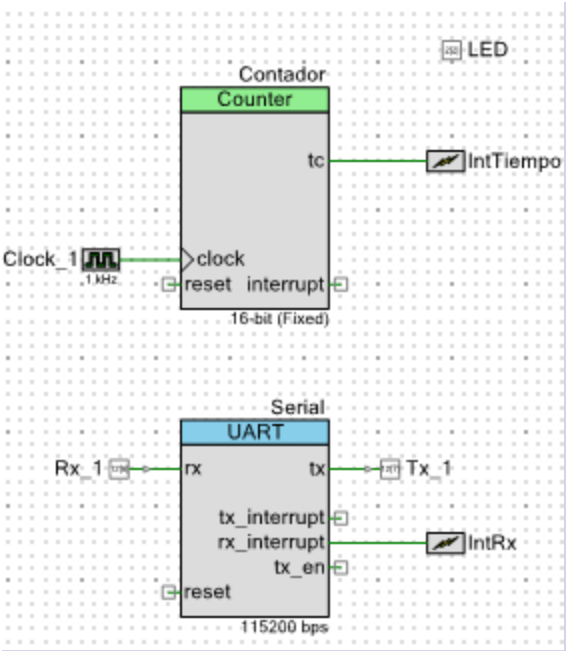

```
void limpiarLCD(){
      LCD_Position(0,0);
      sprintf(textoLCD,"                    ");
      LCD_PrintString(textoLCD);
      LCD_Position(1,0);
      sprintf(textoLCD,"                    ");
      LCD_PrintString(textoLCD);
      LCD_Position(2,0);
      sprintf(textoLCD,"                    ");
      LCD_PrintString(textoLCD);
      LCD_Position(3,0);
      sprintf(textoLCD,"                    ");
      LCD_PrintString(textoLCD);
}

int main(void)
{
    CyGlobalIntEnable;
    LCD_Start();
    limpiarLCD();
    LCD_Position(0,0);
    LCD_PrintString("Texto");

for(;;) {
      if(interrupcionLED==1) {
            sprintf(texto, "Texto");
            limpiarLCD();
            LCD_Position(1,0);
            sprintf(textoLCD,"Texto");
            LCD_PrintString(textoLCD);
      }
interrupcionLED==0;
}
```

✦ LED PILOTO CON INTERRUPCIÓN CADA SEGUNDO.

https://www.udemy.com/profile/13858616/LAURITSME?tab=collections




```
#include "project.h"
uint8 banderaTiempo=0;
char caracterRx;

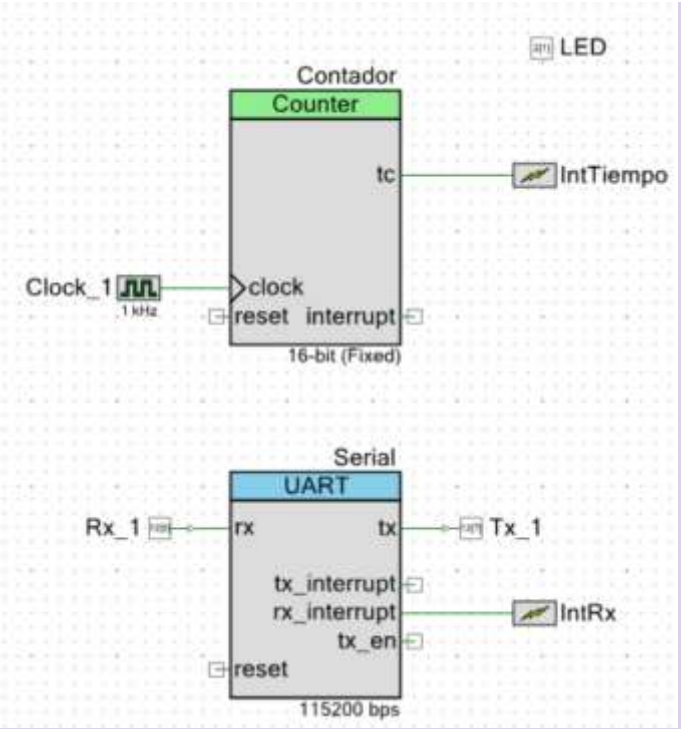
CY_ISR(UnSegundo){
    banderaTiempo=1;
}

int main(void){
    CyGlobalIntEnable;
    Contador_Start();
    IntTiempo_StartEx(UnSegundo);
    Serial_Start();

    for(;;){
        if(banderaTiempo==1){
            LED_Write(!LED_Read());
            banderaTiempo=0;
        }
    }
}
```

✦ ENCENDER UN MOTOR CON DOS COMANDOS.

https://www.udocz.com/profile/138586/LAURITSME?tab=collections




```
#include "project.h"
uint8 banderaTiempo=0;
uint8 banderaDatoNuevo=0;
char caracterRx;

CY_ISR(UnSegundo){
      banderaTiempo=1;
}

CY_ISR(RxData)
{
      banderaDatoNuevo=1;
      caracterRx=Serial_GetChar();
}

int main(void){
      CyGlobalIntEnable;
      Contador_Start();
      IntTiempo_StartEx(UnSegundo);
      Serial_Start();       //
      IntRx_StartEx(RxData);

      for(;;){
      if(banderaTiempo==1){
            LED_Write(!LED_Read());
            banderaTiempo=0;
      }

      if(banderaDatoNuevo==1){
            switch(caracterRx){
                  case 'M':
                  Serial_PutString("Motor Encendido\n");
```

https://www.udec2.com/profile/138586/LAURITSME?tab=collections

```
                break;

                case 'm':
                Serial_PutString("Motor Apagado\n");
                break;

                default:
                Serial_PutString("Error!!!\n");
                break;
            }
            banderaDatoNuevo=0;
        }

        }
}
```

https://www.udocz.com/profile/138586/LAURITSME?tab=collections